

Revisión a Fondo — Pollito

Código (AUDITA) + Visual/UX (ESTETIK) · 29 de julio de 2026

75% listo

Pollito funciona de punta a punta: menú, las 5 skills, memoria persistente, límites de gasto y una interfaz prolija y consistente — hoy mismo se resolvieron varios ajustes de navegación y contraste visual. No está 100% lista para entregarse como regalo todavía por tres motivos concretos: (1) un bug real de concurrencia que puede duplicar gasto de API con un doble Enter, (2) los nombres personalizados de cada skill siguen pendientes, y (3) todavía no se probó con la key real ni se empaquetó en `.exe`. Nada de esto es grave ni tarda mucho en resolverse.

1. Auditoría de Código



Bugs / errores reales

Sin protección contra enviar mensajes en paralelo

Alta

Qué es: al mandar un mensaje, nada deshabilita el campo de texto ni el botón "Enviar" mientras se espera la respuesta de la API. Un doble Enter o dos clicks rápidos en "Enviar" disparan dos llamadas en paralelo.

Por qué importa: cada llamada de más gasta tokens reales (plata) sin que Sofi se dé cuenta, y como `self.historial` se lee y escribe desde dos hilos a la vez, la conversación puede desordenarse o perder coherencia.

Dónde: `skills/base.py`, métodos `_enviar` / `_llamar_api`.

Acción sugerida: deshabilitar el campo y el botón apenas se envía un mensaje, y reactivarlos recién cuando llega la respuesta (o el error).



Para eliminar

Imagen sin usar: `assets/fondo.png`

Baja

Qué es: se genera en `assets/generar_icono.py` pero ninguna parte de la app la carga (confirmado con búsqueda en todo el código).

Por qué importa: peso muerto en el repo, ya estaba anotado como candidato a borrar en `NOTAS/ajustes pollito.txt`.

Acción sugerida: borrarla si no se piensa usar en otro lado.



Mejoras recomendadas

Conteo de tokens diarios sin protección contra condición de carrera

Media

Qué es: `uso_diario.py` → `registrar_uso()` lee, suma y vuelve a escribir el archivo de tokens del día sin ningún lock.

Por qué importa: como las 5 skills corren su llamada a la API en un hilo propio (sin importar cuál se esté viendo en pantalla), dos respuestas que lleguen casi al mismo tiempo pueden pisarse la actualización y subcontar el gasto real — debilita el límite diario de seguridad (`LIMITE_TOKENS_DIARIOS`).

Acción sugerida: proteger la lectura-modificación-escritura con un lock (similar al que ya se usa para el cliente de Anthropic).

Se puede borrar toda la memoria de una skill sin confirmación Media

Qué es: en `skills/memoria.py` , `_resolver_ruta_segura` permite que la ruta pedida resuelva exactamente a la carpeta raíz de memoria, y `_borrar()` llama `shutil.rmtree()` si es un directorio.

Por qué importa: si el modelo alguna vez pide borrar `/memories` a secas (por error o mala instrucción), se pierde toda la memoria persistente de esa skill de una sola vez, sin aviso ni respaldo.

Acción sugerida: rechazar explícitamente el borrado cuando la ruta resuelta sea la raíz misma.

Sin pruebas automatizadas Media

Qué es: a diferencia de `wallet-control`, pollito no tiene ninguna suite de tests configurada.

Por qué importa: justo las dos partes más sensibles a romperse sin darse cuenta — el límite diario de tokens y la protección contra path traversal en la memoria — no tienen ninguna prueba que las respalde.

Acción sugerida: agregar un puñado de tests con `pytest` para `uso_diario.py` y `skills/memoria.py` , el mismo patrón que ya funcionó bien en `wallet-control` con Jest.

✅ Lo que ya funciona bien

Protección real contra path traversal en `skills/memoria.py` — cada operación valida que la ruta se mantenga dentro de la carpeta de esa skill.

Cliente de Anthropic compartido con double-checked locking correcto — sin condición de carrera ahí.

Manejo de errores específico por tipo (Authentication / Connection / RateLimit / APIStatus / genérico), cada uno con mensaje entendible para Sofi.

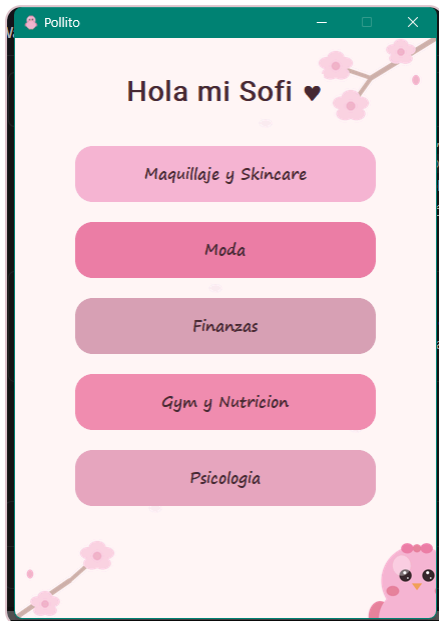
`secreto.py` correctamente en `.gitignore` ; el incidente de la key sin comillas documentado en `NOTAS/` ya está resuelto (verificado que hoy es un string literal válido, sin exponer el valor).

Dependencias instaladas dentro de los rangos fijados en `requirements.txt` (customtkinter 5.2.2, pyinstaller 6.21.0, anthropic 0.72.0, pillow 10.4.0).

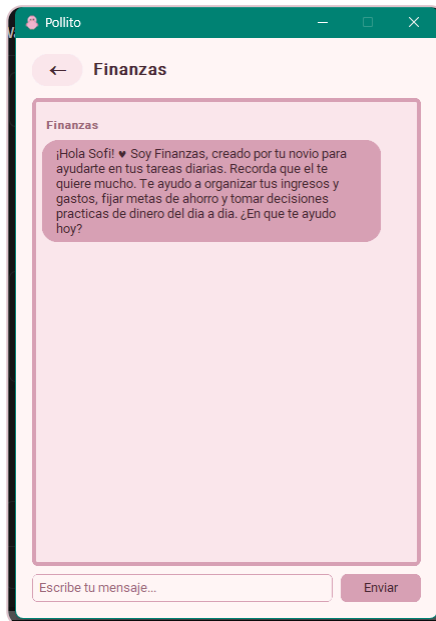
Memoria persistente aislada por skill (una carpeta por skill en `%APPDATA%/Pollito/`), evita mezclar información entre ellas.

Doble control de gasto: límite diario de tokens + límite de mensajes por sesión.

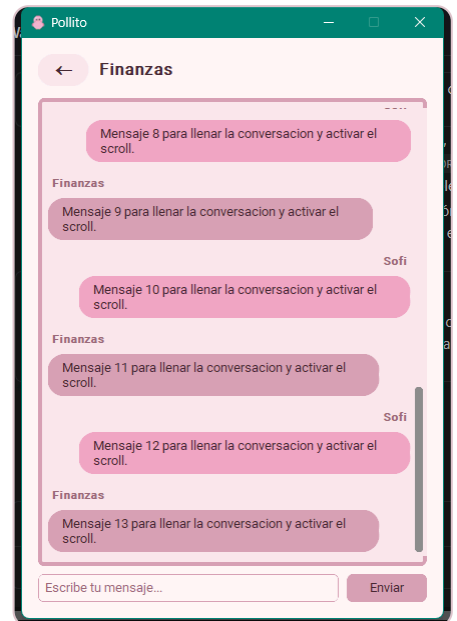
2. Auditoría Visual / UX



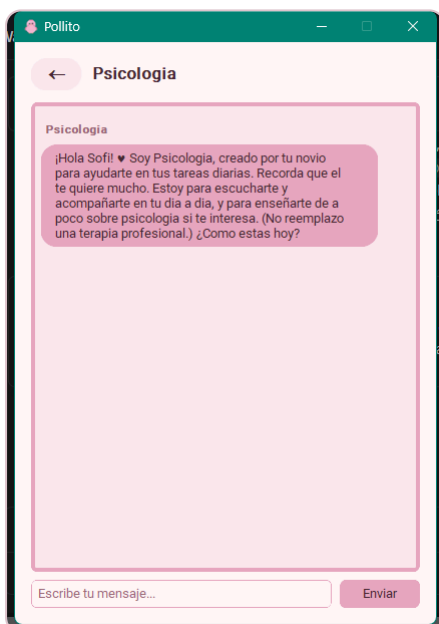
Menú principal



Bienvenida de una skill



Scroll activo (conversación larga)



Otra skill (Psicología)

Accesibilidad

Contraste de texto secundario por debajo del estándar Media

Qué es: medido con la fórmula oficial de WCAG, `TEXTO_SECUNDARIO` (#9C6B7A) sobre `FONDO` (#FFF5F8) da **4.1:1** — el mínimo AA para texto normal es 4.5:1.

Por qué importa: este color se usa en el nombre del remitente arriba de cada burbuja, en los avisos de "Sistema" y en el placeholder del campo de texto — texto secundario en toda la app. No es ilegible, pero no pasa el estándar de accesibilidad.

Acción sugerida: oscurecer un poco `TEXTO_SECUNDARIO` (bajarle luminosidad) hasta pasar 4.5:1.

Fondo de ventana y fondo del recuadro de chat casi idénticos Media

Qué es: FONDO (#FFF5F8) contra FONDO_SECUNDARIO (#FCE8EF) da un contraste de apenas **1.1:1** — prácticamente el mismo color.

Por qué importa: hoy el límite del recuadro se percibe solo gracias al borde de 4px agregado en la sesión de hoy — sin ese borde (o en pantallas con distinto brillo/calibración) el recuadro se "pierde" contra el fondo. Esto fue justamente lo que costó varias rondas de ajuste hoy.

Acción sugerida: separar un poco más los dos tonos de rosa, no depender solo del borde para marcar el límite.

Combinación	Contraste	¿Pasa AA (4.5:1)?
Texto principal sobre fondo	11.56:1	Sí
Texto secundario sobre fondo	4.1:1	No
Texto sobre botón "Enviar"	6.48:1	Sí
Texto sobre acento más bajo (Moda)	4.81:1	Sí
Fondo ventana vs. fondo recuadro de chat	1.1:1	No (uso decorativo, no texto)

Navegación por teclado no verificada Baja

Qué es: no se confirmó si el Tab entre el campo de texto y el botón "Enviar" muestra un indicador de foco visible (CustomTkinter no siempre lo hace por defecto).

Acción sugerida: probarlo a mano y, si no hay indicador claro, agregar un `border_color` distinto en foco.

🌟 Microinteracciones / UX

Sin indicador de "esperando respuesta" Alta

Qué es: al enviar un mensaje no aparece ningún "escribiendo..." ni loading mientras se espera la respuesta real (puede tardar varios segundos, más con búsqueda web activada).

Por qué importa: Sofi no tiene forma de saber si el mensaje se envió — es la otra cara del bug de concurrencia de la sección de código: sin loading state, es mucho más probable que reenvíe "por las dudas".

Acción sugerida: mostrar una burbuja tipo "..." o deshabilitar el input con un texto de "esperando respuesta" hasta que llegue.

📱 Responsive

No aplica en el sentido clásico — la ventana es de tamaño fijo por decisión de diseño confirmada hoy mismo (no redimensionable, no se puede maximizar). El contenido se ve bien dentro de las dimensiones fijas (420×580) en las cuatro capturas de arriba.

✅ Lo que ya funciona bien

Paleta de un solo tono (rosas) coherente en las 5 skills, con variación sutil por acento — se ve como un solo sistema, no cinco estilos distintos.

Navegación de una sola ventana (menú → skill → botón "Atrás"), sin ventanas duplicadas ni saltos de tamaño.

Barra de scroll aparece solo cuando la conversación no entra completa, sin dejar rastro visual cuando está oculta.

Ícono correcto (pollito rosado) tanto en la ventana como en la barra de tareas.

Mascota decorativa y fondo de sakura sutiles, no compiten con la lectura de botones ni texto.

Generado automáticamente el 29 de julio de 2026 por REVISION A FONDO (AUDITA + ESTETIK). No se modificó ningún archivo del proyecto durante esta revisión — código fuente explorado y app real renderizada, sin llamadas a la API real (no se gastaron tokens). Si querés que se arreglen estos hallazgos, PECAS se encarga de los cambios de código/UI y FIXA de los bugs puntuales.